The Theory of Lagrangian Filtering Zones: A Case Study on the Knapsack Constraint

Frédéric Berthiaume[0009−0009−8583−8358]
Claude-Guy Quimper[0000−0002−5899−0217]

Université Laval, Québec, Canada
frederic.berthiaume.1@ulaval.ca, claude-guy.quimper@ift.ulaval.ca

Abstract. CP-based Lagrangian filtering has proven to be a reliable method for enhancing constraint programming solvers. The general theory lacks a uniform framework to explain many known observations. This paper introduces such a framework, along with theoretical results and illustrations. We show why optimal Lagrange multipliers are not always the most effective for filtering. We propose a strategy to move the Lagrange multipliers away from the optimal one to launch a gradient descent that leads to filtering. This paper presents an application of this framework to the versatile Multidimensional Knapsack constraint. The algorithm is tested on two well-known problems : the multidimensional knapsack problem and the uncapacitated facility location problem. The results show a significant speed up compared to the traditional CP-based Lagrangian filtering method on both problems.

Keywords: Constraint programming · Lagrangian relaxation · Knapsack constraint.

1 Introduction

Constraint programming is a sub-branch of Artificial Intelligence interested in combinatorial problems. Constraint solvers get their strength from their filtering algorithms that prune a search tree. Some filtering algorithms encode constraints with a linear program formulation. The reduced cost of each variable can be computed and used for filtering. The Lagrangian relaxation of the constraint's linear program formulation helps to compute efficiently a bound on the objective. This combination of cost-based filtering and Lagrangian relaxation has proven to be a reliable technique in constraint programming [2, 6, 4].

Sellmann [19] shows that optimal Lagrange multipliers, with respect to the objective value, do not provide the best filtering. Suboptimal Lagrange multipliers are more suitable for filtering. Some authors have explored the search for suboptimal Lagrange multipliers that provide more filtering [5, 1, 3]. The general procedure performs Lagrangian optimization and locally changes the Lagrange multipliers. The alteration phase allows for the exploration of new regions in the Lagrange multipliers space, and the discovery of missed filtering.

We have 6 contributions: (1) we propose a unifying framework that explains many known observations of Lagrangian cost-based filtering, (2) we formally define *Lagrangian filtering zones*, (3) we show in which situation optimal Lagrange multipliers do not provide filtering, (4) we present a complete Relaxed($\boldsymbol{\lambda}$) filtering algorithm for the multidimensional Knapsack constraint, (5) we present empirical evidence supporting the theory, (6) we improve the solving times of the knapsack and uncapacitated facility location problems.

2 Background

Column and row vectors are written $\boldsymbol{v}$ and $\boldsymbol{v}^\top$. The dot product of $\boldsymbol{v}$ and $\boldsymbol{w}$ is written $\langle \boldsymbol{v}, \boldsymbol{w} \rangle$. The norm of a vector is written $\|\boldsymbol{v}\| = \sqrt{\langle \boldsymbol{v}, \boldsymbol{v} \rangle}$. A matrix is written M, with the j-row $\boldsymbol{M_{j*}}$ and the i-th column $\boldsymbol{M_{*i}}$, and the components m_{ji}. $\boldsymbol{\nabla} f(\boldsymbol{x})$ is the gradient of f at $\boldsymbol{x}$. The cardinality of a set A is $|A|$.

2.1 0-1 Multidimensional Knapsack Problem and Constraint

The multidimensional knapsack problem is defined as

$$V = \max\{\langle \boldsymbol{v}, \boldsymbol{x} \rangle : \mathrm{W}\boldsymbol{x} \leq \boldsymbol{c}, \boldsymbol{x} \in \{0,1\}^n\}, \tag{1}$$

where $\boldsymbol{v} \in \mathbb{Z}^n$, $\boldsymbol{c} \in \mathbb{Z}^m$ and $\mathrm{W} \in \mathbb{Z}^{m \times n}$. In (1), there are n items and each consumes m resources. The value V is the optimal value.

The multidimensional knapsack constraint $\mathrm{MK}(\boldsymbol{X}, P; \boldsymbol{v}, \mathrm{W}, \boldsymbol{c})$ is a constraint on n binary variables X_i and an integer variable P that is satisfied when:

$$\mathrm{MK}(\boldsymbol{X}, P; \boldsymbol{v}, \mathrm{W}, \boldsymbol{c}) \iff \langle \boldsymbol{v}, \boldsymbol{X} \rangle \geq P \wedge \mathrm{W}\boldsymbol{X} \leq \boldsymbol{c}. \tag{2}$$

Enforcing domain consistency on MK is NP-hard, since deciding if there exists a $\boldsymbol{X}$ for any value in the domain of P that satisfies (2) is NP-complete; [8, 12].

2.2 Cost-Based Filtering

Before 1999, the filtering for (2) was performed by incremental updates of the lower bound of P. This filtering provides a poor relation between the values of the binary variables $\boldsymbol{X}$ and the value of P. Focacci et al. [9] present a new approach to design filtering algorithms to overcome this problem. They introduce the linear relaxation of (1) within the filtering algorithm. Any V coming out of (1) and the linear relaxation of (1) is a valid upper bound of P. The value

$$V[x_i = a] = \max\{\langle \boldsymbol{v}, \boldsymbol{x} \rangle : \mathrm{W}\boldsymbol{x} \leq \boldsymbol{c}, x_i = a, \boldsymbol{x} \in \{0,1\}^n\} \tag{3}$$

and the linear relaxation of (3) are also valid upper bounds of P for any $a \in \mathrm{dom}(X_i)$. Focacci et al. [9] propose to use the bounds $V[x_i = a]$ to detect filtering. Cost-based filtering removes a value a from a domain $\mathrm{dom}(X_i)$ whenever:

$$V[x_i = a] < \min(\mathrm{dom}(P)). \tag{4}$$

Drawback 1 *The time needed to compute the bounds* (3)*, each time the filtering algorithm is called, can hinder the overall performance.*

2.3 Cost-Based-Lagrangian-Filtering

Sellman and Fahle [20] adapt the pioneer work of Held and Karp [10] to Focacci et al. [9] method, and solved a Lagrangian relaxation of (1) like this one

$$V'(\boldsymbol{\lambda}) = \max\{\langle \boldsymbol{v}, \boldsymbol{x}\rangle + \langle \boldsymbol{\lambda}, \boldsymbol{c} - \mathrm{W}\boldsymbol{x}\rangle : \forall\, i,\, x_i \in \mathrm{dom}(X_i)\}. \tag{5}$$

$V'(\boldsymbol{\lambda})$ gives, for any Lagrange multipliers $\boldsymbol{\lambda} \geq \mathbf{0}$, a valid upper bound on P. With this Lagrangian relaxation, the optimization problem is *separable*:

$$V'(\boldsymbol{\lambda}) = \textstyle\sum_{i=1}^{n} \max\{(v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda}\rangle)x_i : x_i \in \mathrm{dom}(X_i)\} + \langle \boldsymbol{\lambda}, \boldsymbol{c}\rangle\,, \tag{6}$$

making the computation of $V'(\boldsymbol{\lambda})$ trivial; resolving the Drawback 1. Equation (6) means that any x_i, associated with a non-fixed variable X_i, is determined by

$$x_i = \begin{cases} 1, & \text{if } v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda}\rangle > 0 \\ 0, & \text{otherwise} \end{cases}. \tag{7}$$

Let $\bar{\boldsymbol{x}}$ be the solution of (6) with components given by (7). With the Lagrangian relaxation (5), $V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda})$ is easier to compute, because the reduced cost of x_i is $V'(\boldsymbol{\lambda}) - V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) = |v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda}\rangle|$, which gives this equation

$$V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) = V'(\boldsymbol{\lambda}) - |v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda}\rangle|. \tag{8}$$

Not every valid Lagrange multipliers vector provides either a good estimation of the upper bound of P or filtering. The bound is tightened by minimizing V' :

$$\min\{V'(\boldsymbol{\lambda}) : \boldsymbol{\lambda} \geq \mathbf{0}\}. \tag{9}$$

Drawback 2 *The function V' is not smooth (there are points where the gradient of the function is not defined), making it challenging to optimize.*

2.4 Subgradient Descent, a More General Gradient Descent

The next definition provides an alternative to the classical gradient when a function is not differentiable.

Definition 1 (Subgradient). *Let $I \subset \mathbb{R}^n$ be an open convex subset of $\mathbb{R}^n$, let $f : I \to \mathbb{R}$ be a convex piecewise linear function, and let $\boldsymbol{x}, \boldsymbol{x_0} \in I$. A* subgradient *of f at $\boldsymbol{x_0}$ is a vector $\mathbb{f}(\boldsymbol{x_0}) \in \mathbb{R}^n$ that satisfies*

$$f(\boldsymbol{x}) - f(\boldsymbol{x_0}) \geq \langle \mathbb{f}(\boldsymbol{x_0}), \boldsymbol{x} - \boldsymbol{x_0}\rangle\,, \forall \boldsymbol{x} \in I$$

i.e., $\mathbb{f}(\boldsymbol{x_0})$ generates a plane $p(\boldsymbol{x}) = \langle \mathbb{f}(\boldsymbol{x_0}), \boldsymbol{x} - \boldsymbol{x_0}\rangle + f(\boldsymbol{x_0})$ tangent to f at $\boldsymbol{x_0}$.

Sellmann and Fahle [20] propose a filtering algorithm based on a *subgradient descent* to solve (9). The algorithm builds a sequence $(\boldsymbol{\lambda^{(k)}})_{k\geq 0}$ with

$$\boldsymbol{\lambda^{(k+1)}} = \max\left(\mathbf{0}, \boldsymbol{\lambda^{(k)}} - \alpha_k \mathbb{v}(\boldsymbol{\lambda^{(k)}})\right), \tag{10}$$

where $\max(\cdot)$ acts as the projector on $\mathbb{R}^m_+$ (the valid Lagrange multipliers), α_k is a step size, and $\mathbb{v}(\boldsymbol{\lambda^{(k)}})$ is a subgradient. To ensure that $(\boldsymbol{\lambda^{(k)}})_{k\geq 0}$ converges to a minimizer $\boldsymbol{\lambda}^\star$ of V', one can use Polyak [17] step size's

$$\alpha_k = \frac{\beta_k(V'(\boldsymbol{\lambda^{(k)}}) - \min(\mathrm{dom}(P)))}{\|\mathbb{v}(\boldsymbol{\lambda^{(k)}})\|^2},\ 0 < \beta_k < 2. \tag{11}$$

2.5 Sellmann's Sub-Optimality Observation

Sellmann and Fahle [20] notice that testing (4) for each Lagrange multipliers vector in $(\boldsymbol{\lambda^{(k)}})_{k\geq 0}$ provides stronger filtering than testing (4) at the end of the sequence. Then, Sellmann [19] proves that the optimal Lagrange multipliers for (9) are not optimal for filtering. Essentially, minimizing $V'[x_i = 1 - \bar{x}_i]$ is not the same task as minimizing V'. Sellmann [19] concludes that there are no reasons to discard suboptimal Lagrange multipliers to do cost-based filtering.

Drawback 3 *Sellmann [19] is a paramount observation, but it does not provide any reason as to why the Lagrange multipliers in* $(\boldsymbol{\lambda^{(k)}})_{k\geq 0}$ *are better at filtering.*

2.6 Oscillations of the Objective Function

Inspired by Sellmann [19], Isoart and Régin [11] compare the variation of V' and the amount of filtering detected during the generation of $(\boldsymbol{\lambda^{(k)}})_{k\geq 0}$. They notice that in the early steps the value of V' *oscillates* a lot and more filtering is detected. They suggest that only the early terms in the sequence lead to filtering and that the optimization of (9) should be stopped. Their method is faster by a factor of two than the state-of-the-art; supporting Sellmann [19]'s observation.

2.7 Local Alterations Algorithm

Boudrault and Quimper [5] present a local alterations cost-based Lagrangian filtering algorithm for the WeightedCircuit constraint. Their algorithm locally explores the Lagrange multipliers space, *while generating the sequence* $(\boldsymbol{\lambda^{(k)}})_{k\geq 0}$, to find optimal filtering Lagrange multipliers, by exploiting the properties of a weighted graph. There is no guarantee that all filtering is applied, but they obtain a good speed up against the state-of-the-art in CP.

Berthiaume and Quimper [3] propose a local alterations filtering algorithm for the AtMostNValue constraint. In contrast to Boudrault and Quimper [5], the algorithm makes many subgradient descents (potentially one for each binary variable) that start from $\boldsymbol{\lambda}^\star$, the minimizer of V', and minimize $V'[x_i = 1 - \bar{x}_i]$. To avoid too many time-consuming gradient descents that do not lead to filtering, the descent is performed for a variable x_i only if, for a given threshold τ, this inequality holds: $|V'[x_{i'} = 1 - \bar{x}_{i'}](\boldsymbol{\lambda}^\star) - \min(\mathrm{dom}(P))| < \tau$; i.e., the algorithm minimizes the functions $V'[x_i = 1 - \bar{x}_i]$ that are close to detect filtering.

Berthiaume and Quimper [3] show that the functions $V'[x_i = 1 - \bar{x}_i]$ are generally neither convex nor concave over all $\mathbb{R}^m$, but when restricted to:

$$Q_{i1} = \{\boldsymbol{\lambda} : v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda}\rangle \geq 0\} \tag{12}$$

$$Q_{i0} = \{\boldsymbol{\lambda} : v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda}\rangle \leq 0\}, \tag{13}$$

the functions $V'[x_i = 1 - \bar{x}_i]$ are convex. Q_{i1} and Q_{i0} are the sets of Lagrange multipliers where the variable $x_i = 1$ and $x_i = 0$ in (7). They also show that the gradient of the functions V' and $V'[x_i = 1 - \bar{x}_i]$ is discontinuous on the set

$$\textstyle\bigcup_{i=1}^{n}\{\boldsymbol{\lambda} : v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda}\rangle = 0\}. \tag{14}$$

Berthiaume and Quimper [3] compare their algorithm with Cambazard and Fages [6]' algorithm. They obtain a significant speed up. They also observe that:

Observation 1 *The algorithm does not cycle due to a decreasing step size.*

Drawback 4 *Their algorithm has three drawbacks.*

1. *They mention the term* filtering zone *without a formal definition;*
2. *They do not check if $\boldsymbol{\lambda}^\star$ is a good point for the threshold test;*
3. *There is no guarantee that every filtering is applied.*

2.8 The Relaxed($\boldsymbol{\lambda}$) Consistency

Sellmann [19] introduces the Relaxed($\boldsymbol{\lambda}$) consistency that filters a value from a domain whenever there exist Lagrange multipliers that can filter it.

Definition 2. *Let $C(\boldsymbol{X}, P)$ be a constraint whose solution space is given by the linear relation $\langle \boldsymbol{c}, \boldsymbol{X} \rangle \geq P \wedge \mathrm{A}\boldsymbol{X} \leq \boldsymbol{b}$. Let $v \in \mathrm{dom}(X_j)$ and $B'[x_j = v]$ be the relaxed upper bound of P:*

$$B'[x_j = v](\boldsymbol{\lambda}) = \max \left\{ \left\langle \boldsymbol{c} - \mathrm{A}^\top \boldsymbol{\lambda}, \boldsymbol{x} \right\rangle : \forall i,\, x_i \in \mathrm{dom}(X_i), x_j = v \right\} + \langle \boldsymbol{\lambda}, \boldsymbol{b} \rangle .$$

The constraint $C(\boldsymbol{X}, P)$ is Relaxed($\boldsymbol{\lambda}$)*-consistent if and only if*

$$\forall i,\, \forall v \in \mathrm{dom}(X_i),\, \forall \boldsymbol{\lambda} \in \mathbb{R}_+^m \quad B'[x_i = v](\boldsymbol{\lambda}) \geq \min(\mathrm{dom}(P)). \tag{15}$$

3 Contribution

Existing Lagrangian based methods operate without an explicit description of the structure of the Lagrange multiplier space. This paper provides the missing theoretical foundation needed to understand the behavior of most CP-based Lagrangian filtering approaches. We introduce new theoretical concepts. We provide an illustration of these concepts with an instance of a MK constraint.

3.1 New Interpretation of the Lagrange Multipliers Space

Example 1. Consider this instance of a MK constraint:

$$\begin{aligned} \#\text{items } n &= 4 & \#\text{resources } m &= 2 \\ \boldsymbol{v} &= \begin{pmatrix} 10 & 16 & 20 & 8 \end{pmatrix}^\top & \boldsymbol{c} &= \begin{pmatrix} 5 & 5 \end{pmatrix}^\top \\ \mathrm{W} &= \begin{pmatrix} 1 & 1 & 3 & 3 \\ 4 & 3 & 2 & 1 \end{pmatrix} & \mathrm{dom}(P) &= [32, 60]. \end{aligned}$$

There are two resource constraints, so $\boldsymbol{\lambda} \in \mathbb{R}^2$. The valid Lagrange multipliers are in the positive quadrant of Figure 1.

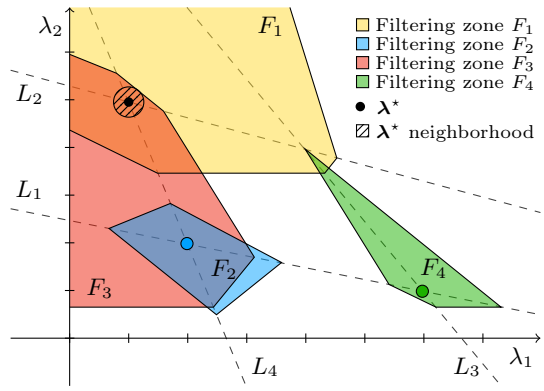

Fig. 1. Lagrange multipliers space.

Definition 3. *The set* $L_i = Q_{i1} \cap Q_{i0}$ *is called the* decision line *of the variable* x_i*, because* L_i *separates the sets* Q_{i1} *and* Q_{i0}*.*

Note 1. Observed that $L_i = \{\boldsymbol{\lambda} : v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda} \rangle = 0\}$.

In Example 1, there are four items, hence there are four decision lines, one for each variable, and are drawn with dashed lines in Figure 1. The functions are not differentiable on the decision lines. We see immediately that the decision lines create regions in the Lagrange multipliers space. In these regions, the signs of the coefficients $v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda} \rangle$ of each variable remain the same. That is, each Lagrange multipliers vector in the *open interior* of a region enclosed by decision lines produces the same $\bar{\boldsymbol{x}}$.

Definition 4. *Let* $F_i = \{\boldsymbol{\lambda} \in \mathbb{R}_+^m : V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) < \min(\mathrm{dom}(P))\}$ *be the* filtering zone *of* X_i*; also known as the level set of* $V'[x_i = 1 - \bar{x}_i]$*.*

If it exists, any Lagrange multipliers vector $\boldsymbol{\lambda} \in F_i$ satisfies the reduced cost filtering condition (4); which means we can add the value $1 - \bar{x}_i$ to the values to be filtered out from $\mathrm{dom}(X_i)$. Hence, if the filtering zone F_i is non-empty, then filtering can be detected. The filtering zone F_i is a function of the lower bound of P and if $\min(\mathrm{dom}(P)) < \min\{V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) : \boldsymbol{\lambda} \in \mathbb{R}_+^m\}$ then no filtering is detected for any $\boldsymbol{\lambda} \in \mathbb{R}_+^m$ during the call to the filtering algorithm. So as the solver increases $\min(\mathrm{dom}(P))$ the filtering zones increase in size. Definition 4 is a formal definition, hence resolving Drawback 4.1. The filtering zones of the variables in Example 1 are drawn in Figure 1.

Hence, the goal of every Lagrangian cost-based filtering algorithm should be to : 1. find a good bound for P and 2. find Lagrange multipliers that are in the filtering zones $F_1, F_2, ..., F_n$. A subgradient descent is a natural choice for the first objective because of the Lagrangian theory. For the second objective, a *unique* subgradient descent might not be the best way to achieve this goal.

Drawback 3 clearly states that there are optimal filtering Lagrange multipliers. In Figure 1, the blue and green dots are optimal for the variables X_2 and X_4, i.e., they minimize $V'[x_2 = 1 - \bar{x}_2]$ and $V'[x_4 = 1 - \bar{x}_4]$. By using a *unique* subgradient descent, it is very unlikely that any of the optimal filtering Lagrange multipliers will be visited. On the other hand, the sequence of Lagrange multipliers generated by a subgradient descent (9) may contain some Lagrange multipliers in some filtering zones; formalizing the sub-optimality observation.

If we use *many* subgradient descents, at least one in each $Q_{i1} \cap \mathbb{R}^m_+$ and $Q_{i0} \cap \mathbb{R}^m_+$ for all variables, then if a filtering Lagrange multipliers vector exists, we are sure to detect filtering, since $V'[x_i = 1 - \bar{x}_i]$ is a convex function on Q_{ia}.

In the new geometrical interpretation, Isoart and Régin [11]'s observation arises naturally. From Definition 4, we know that filtering zones lie in the Lagrange multipliers space, some are bounded and some not, some cluster together, and some are relatively isolated. The point is that a vast cover of the Lagrange multipliers space is more likely to detect filtering than a small neighborhood around $\boldsymbol{\lambda}^\star$. Furthermore, the objective value V' will vary much more on the vast cover of the Lagrange multipliers space than within a small neighborhood around $\boldsymbol{\lambda}^\star$. Also, the filtering within the small neighborhood will likely be redundant. From this analysis, big variations are a sign of a big cover of $\mathbb{R}^m_+$ which generally leads to better filtering than within a small neighborhood of $\boldsymbol{\lambda}^\star$.

The key insight is that using many subgradient descents appears to be a natural building block for Lagrangian cost-based filtering algorithms. A subgradient descent is used to get a good bound on P. The rest of them are to find filtering zones. The first one naturally appears from the Lagrangian optimization. The filtering subgradient descents are not straightforward without the idea of filtering zones. These subgradient descents do not aim at an optimal filtering $\boldsymbol{\lambda}$, but rather for the filtering zone; hence, they can be stopped rapidly.

3.2 Is $\boldsymbol{\lambda}^\star$ a good starting point to apply the threshold test?

We note some observations from Figure 1 that we later prove. These observations are at the core of the new filtering algorithm we propose. In Figure 1, the optimal Lagrange multipliers $\boldsymbol{\lambda}^\star$ lie at the intersection of two decision lines; L_2 and L_4.

Since the function V' is piece-wise linear and the solution to problem (1) is non trivial[1], $\boldsymbol{\lambda}^\star$ can only be in three positions: (1) at the intersection of decision lines, (2) on a segment of a decision line, or (3) in a section enclosed by decision lines. In cases (2) and (3), there would be many optimal Lagrange multipliers.

Definition 5. *Let $\Lambda^\star = \{\boldsymbol{\lambda} : V'(\boldsymbol{\lambda}) = V'(\boldsymbol{\lambda}^\star)\}$ be the set of optimal Lagrange multipliers.*

In Figure 1, $\Lambda^\star = \{\boldsymbol{\lambda}^\star\}$. In all cases (1), (2) and (3), there are always some decision lines intersecting $\Lambda^\star$.

Definition 6. *Let $\mathcal{J} = \{j : L_j \cap \Lambda^\star \neq \emptyset\}$ be the set of indices of decision lines intersecting the optimal Lagrange multipliers $\Lambda^\star$.*

[1] Trivial solutions are $\boldsymbol{x} = \mathbf{1}$ and $\boldsymbol{x} = \mathbf{0}$ for which $\boldsymbol{\lambda}^\star = \mathbf{0}$ and $\boldsymbol{\lambda}^\star$ is unbounded.

In Figure 1, $\mathcal{J} = \{2, 4\}$ and note that the only filtering zones that do not contain $\boldsymbol{\lambda}^\star$ are F_2 and F_4. This suggests that there is a relation between the position of $\boldsymbol{\lambda}^\star$, with respect to the decision lines, and the filtering zones. We could even say that $\boldsymbol{\lambda}^\star$ are bad Lagrange multipliers for filtering or even for applying the threshold test for the variables X_2 and X_4, but not for the filtering of X_1 and X_3. This section formalizes these observations.

Definition 7. *Let* $T_i(\tau) = \{\boldsymbol{\lambda} \in \mathbb{R}_+^m : V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) < \min(\mathrm{dom}(P)) + \tau\}$ *be the threshold zone of the variable* X_i *with threshold* τ.

Threshold zones are bloated filtering zones and are not drawn in Figure 1.

Lemma 1. *If filtering can be detected* ($F_i \neq \emptyset$)*, then* $T_i(\tau) \neq \emptyset$ *for any* $\tau \geq 0$.

Proof. It follows from Definitions 4 and 7, that $\forall \tau \geq 0$ we have $F_i \subset T_i(\tau)$. So by the assumption that $F_i \neq \emptyset$, then $T_i(\tau) \neq \emptyset$. □

In Figure 1, all filtering zones are present (hence non-empty), so any bloated version of these zones, like the threshold zones $T_i(\tau)$, are also non-empty.

Lemma 2. $\exists \tau > 0$ *such that the threshold zone is non-empty; i.e.,* $T_i(\tau) \neq \emptyset$.

Proof. $\forall \tau > \min_{\boldsymbol{\lambda} \in \mathbb{R}_+^m} \{V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda})\} - \min(\mathrm{dom}(P))$, $T_i(\tau) \neq \emptyset$. □

Suppose that, in Figure 1, the filtering zone F_4 was empty, which means that no filtering could be detected. The green dot, the minimizer of $V'[x_4 = 1 - \bar{x}_4]$ in $\mathbb{R}_+^m$, would still be there. Hence, for any threshold $\tau > V'[x_4 = 1 - \bar{x}_4](\boldsymbol{\lambda}) - \min(\mathrm{dom}(P))$, the threshold zone $T_4(\tau)$ would be non-empty. This is a case where the algorithm would trigger an unfruitful descent that leads to no filtering.

Theorem 1. *If the current problem has a non-trivial solution, then there is a non-instantiated variable* X_i*, such that some optimal Lagrange multipliers lie on its decision line* L_i*, in other words,* $\exists i$ *such that* $\Lambda^\star \cap L_i \neq \emptyset$.

Proof. Since the current problem has a non-trivial solution, we know that $\boldsymbol{\lambda}^\star \in \Lambda^\star$ implies that $\mathcal{J} \neq \emptyset$ and by Definitions 5 and 6, $\Lambda^\star = \bigcap_{j \in \mathcal{J}} Q_{j\bar{x}_j}$. Hence, the theorem holds for all $i \in \mathcal{J}$. □

As mentioned before, Theorem 1 is a consequence of V' being piece-wise linear. The implication is now clear: as long as there are non-instantiated variables, there are decision lines intersecting $\Lambda^\star$. With Theorem 1, the observation that $\boldsymbol{\lambda}^\star$ is at the intersection of the decision lines L_2 and L_4 in Figure 1 is not surprising.

Lemma 3. $\forall \boldsymbol{\lambda} \in L_i$*, we have* $V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) = V'(\boldsymbol{\lambda})$.

Proof. From Note 1, $\forall \boldsymbol{\lambda} \in L_i$, $V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) = V'(\boldsymbol{\lambda}) - |v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda} \rangle| = V'(\boldsymbol{\lambda}) - 0 = V'(\boldsymbol{\lambda})$. □

From the definition of $V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) = V'(\boldsymbol{\lambda}) - |v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda} \rangle|$ for any $\boldsymbol{\lambda} \notin L_i$, $V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) < V'(\boldsymbol{\lambda})$. We also know that $V'[x_i = 1 - \bar{x}_i]$ decreases at most linearly with the distance to the decision lines L_i. Hence, in Figure 1, it is not surprising that the filtering zones are far from their decision lines (e.g., F_1 is far from L_1). For the following, let $\tau_0 = V'(\boldsymbol{\lambda}^\star) - \min(\mathrm{dom}(P))$.

Theorem 2. *Any $\boldsymbol{\lambda}$ that is optimal and on a decision line L_i ($\boldsymbol{\lambda} \in \Lambda^\star \cap L_i$) is in the threshold zone of the variable X_i ($\boldsymbol{\lambda} \in T_i(\tau)$) if and only if $\tau > \tau_0$.*

Proof. [$\Rightarrow$] If $\boldsymbol{\lambda} \in T_i(\tau)$, then $V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) < \min(\mathrm{dom}(P)) + \tau$. Lemma 3 and Definition 5 ($\Lambda^\star$) ensure that $V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) = V'(\boldsymbol{\lambda}) = V'(\boldsymbol{\lambda}^\star)$. So, $V'(\boldsymbol{\lambda}^\star) < \min(\mathrm{dom}(P)) + \tau$, hence $\tau_0 < \tau$.

[$\Leftarrow$] We still have $V'(\boldsymbol{\lambda}^\star) = V'(\boldsymbol{\lambda}) = V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda})$, so $\tau > \tau_0 = V'(\boldsymbol{\lambda}^\star) - \min(\mathrm{dom}(P)) = V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) - \min(\mathrm{dom}(P))$, meaning $\boldsymbol{\lambda} \in T_i(\tau)$. □

In Figure 1, $V'(\boldsymbol{\lambda}^\star) = 37$ and $\min(\mathrm{dom}(P)) = 32$, hence for any threshold $\tau > \tau_0 = 37 - 32 = 5$, $\boldsymbol{\lambda}^\star$ is in the threshold zone of every variable. However, if the threshold $\tau \leq 5$, then $\boldsymbol{\lambda}^\star$ would not be in the threshold zones $T_2(\tau)$ and $T_4(\tau)$. In this case, even if filtering *can* be detected (because the filtering zones are non-empty), the algorithm can *miss* the filtering. This is alarming. The threshold test is supposed to reduce the number of descents that does not lead to filtering, but could prevent the detection of filtering.

Not all is lost, the problematic Lagrange multipliers are only the ones that are optimal and on a decision line. Consider $j \in \mathcal{J}$, since $V'[x_j = 1 - \bar{x}_j]$ deteriorates at most linearly with the distance to the decision line L_j, we know that any $\boldsymbol{\lambda}$ in a neighborhood of $\boldsymbol{\lambda}^\star$ such that $\boldsymbol{\lambda} \notin L_j$ is more likely to pass the threshold test. One can consider the subgradient descents as projectors on the non-empty filtering zones. By performing the threshold test after a few steps of the gradient descent, rather than at the very beginning, the algorithm makes a more educated guess as to whether there is a filtering zone or not. Furthermore, performing more steps before the threshold test should lessen the impact of the parameter τ on the overall performance since, at each step, we get closer to the filtering zone.

Hence, to answer the question in the title of the section, $\boldsymbol{\lambda}^\star$ is bad for the threshold test of the variable j if and only if $\boldsymbol{\lambda}^\star$ lies on its decision line L_j.

3.3 Algorithm

Algorithm 1 is our filtering algorithm for the MK constraint. The decision variables subject to the MK constraint are $\boldsymbol{X}$ and P, while the constant parameters are $\boldsymbol{v}, \mathrm{W}$, and $\boldsymbol{c}$, as defined in (1). β_0 and γ_0 are the initial step sizes for the Lagrangian optimization and local alterations. maxItX is the maximal number of steps we perform in our local alteration phase. τ is the threshold. minItX is the number of steps before applying the threshold test. lF is an array of sets of values that were flagged for filtering. lF is short for *Lagrangian forbidden value.* maxIt is the maximal length of the sequence. $\boldsymbol{\lambda}$ is a Lagrange multiplier vector. V' is the relaxed valid upper bound on V in (1) and V_{best} its best known value.

Algorithm 1 filters values using Lagrangian cost-based filtering. First, the function V' is minimized in the repeat loop. Line 5 solves the Lagrangian subproblem. Line 6 computes the upper bound. For all non-instantiated variables, Line 8 performs cost-based filtering by checking the inequality in Definition 4. Line 9 computes the step size. Line 10 updates the Lagrange multiplier. Line 11 stops the minimization of V' if there are maxIt steps or if the greatest difference

Algorithm 1: Propagator for the MK constraint.

1 **Input**: $\boldsymbol{X}, P, \boldsymbol{v}, \mathrm{W}, \boldsymbol{c}, \beta_0, \mathrm{maxIt}, \mathrm{maxItX}, \tau, \gamma_0, \mathrm{minItX}$
2 **Output**: Filtered domains
3 $lF \leftarrow [\emptyset_1, ..., \emptyset_n], \boldsymbol{\lambda} \leftarrow \boldsymbol{0}, k \leftarrow 0,\ \beta \leftarrow 0, \boldsymbol{x} \leftarrow \boldsymbol{0}, V_{best} \leftarrow \infty$
4 **repeat**
5 SolveSubProblem($\boldsymbol{v} - \mathrm{W}^\top \boldsymbol{\lambda}$) //Algorithm 2
6 $V' \leftarrow \langle \boldsymbol{v} - \mathrm{W}^\top \boldsymbol{\lambda}, \boldsymbol{x} \rangle + \langle \boldsymbol{\lambda}, \boldsymbol{c} \rangle$, $V_{best} \leftarrow \min(V', V_{best})$
7 **for** $i = 1$ **to** n **where** $|\,\mathrm{dom}(X_i)| = 2$ **do**
8 **if** $\boldsymbol{\lambda} \in F_i$ **then** $lF[i] \leftarrow lF[i] \cup \{1 - \bar{x}_i\}$.
9 $\beta \leftarrow \frac{\beta_0}{2^{\lfloor k/10 \rfloor}} \frac{(V' - \min(\mathrm{dom}(P)))}{\|\boldsymbol{c} - \mathrm{W}\boldsymbol{x}\|^2},\ \boldsymbol{\lambda'} \leftarrow \boldsymbol{\lambda}$
10 $\boldsymbol{\lambda} \leftarrow \max(\boldsymbol{0}, \boldsymbol{\lambda'} - \beta(\boldsymbol{c} - \mathrm{W}\boldsymbol{x})),\ k \leftarrow k + 1$
11 **until** $(k = \mathrm{maxIt}) \vee (\max_{0 \le j \le m} |\lambda_j - \lambda'_j| \le 10^{-4})$
12 **if** maxItX> 0 **then**
13 **for** $i = 1$ **to** n **where** $|\,\mathrm{dom}(X_i)| = 2$ **do**
14 Update&Flag(i, Q_{i0}) // Algorithm 3
15 Update&Flag(i, Q_{i1})// Algorithm 3
16 **for** $i = 1$ **to** n **do** $\mathrm{dom}(X_i) \leftarrow \mathrm{dom}(X_i) \setminus lF[i]$
17 $\mathrm{dom}(P) \leftarrow \{p \in \mathrm{dom}(P) : p \le V_{best}\}$

between consecutive Lagrange multipliers is smaller than a certain factor. The local alteration phase is triggered on Line 13. For each non-instantiated variable, Lines 14 and 15 search for filtering Lagrange multipliers on both sides of L_i. Algorithms 3 and 1 are similar. The difference is that if the Lagrange multiplier is not in the set Q, then Line 8 calls a quadratic solver to project $\boldsymbol{\lambda}$ onto Q.

Theorem 3. *There exist a threshold τ such that Algorithm 1, with* maxItX $\to \infty$*, enforces* Relaxed($\boldsymbol{\lambda}$) *on* MK.

Proof. With $\tau > \max_{1 \le i \le n}\{V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}^\star)\} - \min(\mathrm{dom}(P))$, the condition on Line 4 of Algorithm 3 is always false; i.e., $\forall i, \boldsymbol{\lambda}^\star \in T_i(\tau)$. Algorithm 3 is guaranteed to converge to a minimizer in $\mathbb{R}^m_+ \cap Q_{i0}$ and in $\mathbb{R}^m_+ \cap Q_{i1}$ with the chosen step size and maxItX $\to \infty$. If the minimizers are in F_i, the value is flagged to be filtered on Line 12 in Aglorithm 3 and removed on Line 16 of Algorithm 1. Hence, $\forall i$ and $\forall v \in \mathrm{dom}(X_i)$, $\forall \boldsymbol{\lambda} \in \mathbb{R}^m_+, V'[x_i = 1 - \bar{x}_i](\boldsymbol{\lambda}) \ge \min(\mathrm{dom}(P))$. □

Theorem 3 ensures that Algorithm 1 can enforce Relaxed($\boldsymbol{\lambda}$).

The computation time is improved by calling Algorithm 3 once and avoiding the quadratic program. We replace, in Algorithm 1, lines 14 and 15 by

Algorithm 2: SolveSubProblem($\boldsymbol{q}$)

1 **for** $i = 1$ **to** n **do**
2 **if** $|\,\mathrm{dom}(X_i)| = 1$ **then** $x_i \leftarrow \min(\mathrm{dom}(X_i))$
3 **else if** $q_i > 0$ **then** $x_i \leftarrow 1$ **else** $x_i \leftarrow 0$

Algorithm 3: Update&Flag(i, Q)

1 **Input**: the index of a variable i, Q a set.
2 save$(\boldsymbol{\lambda}, \boldsymbol{x}, V')$, $k \leftarrow 0$, $\gamma \leftarrow 0$
3 **for** $k = 0$ **to** maxItX $- 1$ **do**
4   **if** $k = \text{minItX} \wedge \boldsymbol{\lambda} \notin T_i(\tau)$ **then break**
5     $q_i \leftarrow v_i - \langle \boldsymbol{W_{*i}}, \boldsymbol{\lambda} \rangle$, $\gamma \leftarrow \frac{\gamma_0}{2^{\lfloor k/5 \rfloor}} \frac{V' - |q_i| - \min(\text{dom}(P))}{\|\boldsymbol{c} - \text{W}\boldsymbol{x} + \text{sgn}\,(q_i)\boldsymbol{W_{*i}}\|^2}$, $\boldsymbol{\lambda'} \leftarrow \boldsymbol{\lambda}$
6     $\boldsymbol{\lambda} \leftarrow \max\left(\boldsymbol{0}, \boldsymbol{\lambda'} - \gamma(\boldsymbol{c} - \text{W}\boldsymbol{x} + \text{sgn}\,(q_i)\boldsymbol{W_{*i}})\right)$
7   **if** $Q \neq \mathbb{R}^m_+$ **and** $\boldsymbol{\lambda} \notin \mathbb{R}^m_+ \cap Q$ **then**
8         $\boldsymbol{\lambda} \leftarrow \text{argmin}\{\|\boldsymbol{\lambda} - \boldsymbol{\lambda'}\|^2 : \boldsymbol{\lambda'} \in \mathbb{R}^m_+ \cap Q\}$
9   SolveSubProblem$(\boldsymbol{v} - \text{W}^\top \boldsymbol{\lambda})$ //Algorithm 2
10     $V' \leftarrow \left\langle \boldsymbol{v} - \text{W}^\top \boldsymbol{\lambda}, \boldsymbol{x} \right\rangle + \langle \boldsymbol{\lambda}, \boldsymbol{c} \rangle$
11   **if** $\boldsymbol{\lambda} \in F_i$ **then**
12         $lF[i] \leftarrow lF[i] \cup \{1 - \bar{x}_i\}$
13   **break**
14 restore$(\boldsymbol{\lambda}, \boldsymbol{x}, V')$

Update&Flag$(i, \mathbb{R}^m_+)$. Due to Observation 1, the Algorithm 1 does not cycle. We observe empirically that the cases where this modification fails to filter are rare.

4 Experiments & Results

4.1 Multidimensional Knapsack Problem

We implemented our filtering algorithm to test it over instances of the multidimensional knapsack problem. We used three benchmarks for our experiments from Beasley's library[2], which has been widely used to test the performance of different algorithms Lu et al. [14]. Chu and Beasley [7] randomly generated 9 classes of 30 instances. All instances in a given class have the same number of resources and items. They chose 5, 10, and 30 resources, and 100, 250, and 500 items. We call this benchmark Chu and Beasley [7]'s instances.

The second benchmark from Beasley's library contains 48 instances drawn from the literature. We call this benchmark: Beasley's instances.

The third benchmark contains seven instances designed by Petersen [15]. We call this benchmark Petersen [15]'s instances.

We implemented Algorithm 1 in Java in the Choco solver [18]. The code and benchmarks are available on GitHub[3]. We use the quadratic solver ojAlgo [16] to implement Line 8 of Algorithm 3. The experiments were carried out on a MacBook Pro with a M2 chip and 8 Gb of RAM. $\beta_0 = 5.0$ and $\gamma_0 = 15.0$, maxIt $= 1000$ is the maximal number of steps in Algorithm 1 and maxItX $= 60$ the maximum number of additional steps in the local alterations phase. The

[2] https://people.brunel.ac.uk/~mastjjb/jeb/orlib/mknapinfo.html
[3] https://github.com/frbert3/knapsackConstraintCPAIOR2026

#	Time [seconds]						Nodes [-]					
	LIN	LR	LR+0	LR+1	LR+2	LR+3	LIN	LR	LR+0	LR+1	LR+2	LR+3
0	67.73	–	–	0.81	**0.70**	0.76	2×10^6	–	–	857	**720**	765
1	–	–	–	**5.51**	5.61	6.45	–	–	–	6946	7106	**6686**
8	41.95	44.84	40.20	47.82	**22.51**	22.93	5×10^6	68928	68928	65602	**39953**	40684
9	–	3.21	3.08	3.33	5.84	**1.54**	–	4754	4754	3897	7062	**2093**
18	0.47	–	–	**0.01**	**0.01**	**0.01**	80255	–	–	**39**	**39**	**39**
19	2.52	5.19	4.93	1.01	0.80	**0.70**	4×10^5	12913	12913	2660	2153	**1909**
20	**0.57**	–	–	9.71	7.81	4.44	84632	–	–	23885	18628	**11441**
21	2.75	75.61	61.07	**0.04**	**0.04**	**0.04**	4×10^5	2×10^5	2×10^5	101	**91**	**91**
22	2.25	–	–	1.52	1.36	**1.29**	3×10^5	–	–	3860	3495	**3214**
23	51.98	–	–	**0.06**	**0.06**	0.07	7×10^6	–	–	**127**	131	131
24	10.13	–	–	1.11	1.21	**0.98**	1×10^6	–	–	2734	2487	**2287**
25	–	–	–	5.51	4.37	**2.36**	–	–	–	2251	**2077**	2513
26	118.54	–	–	**0.04**	**0.04**	0.05	2×10^7	–	–	**92**	**92**	**92**
27	–	0.15	0.14	**0.13**	**0.13**	**0.13**	–	529	520	412	**410**	**410**
28	–	–	–	0.13	**0.12**	0.13	–	–	–	377	361	**360**
29	–	–	–	3.00	**1.89**	2.40	–	–	–	5021	**3061**	3879

#	Time [seconds]						Nodes [-]					
	LIN	LR	LR+0	LR+1	LR+2	LR+3	LIN	LR	LR+0	LR+1	LR+2	LR+3
30	–	–	–	38.90	37.02	**31.80**	–	–	–	65178	60766	**48460**
31	–	–	–	0.27	0.23	**0.21**	–	–	–	531	430	**367**
32	–	–	–	**0.81**	0.91	1.04	–	–	–	**1622**	1729	1845
33	–	–	–	**0.09**	0.10	0.11	–	–	–	**198**	202	202
34	–	0.58	0.55	**0.10**	0.11	0.12	–	933	933	**141**	**141**	**141**
35	–	0.21	0.20	**0.13**	0.14	0.15	–	491	491	257	244	**243**
36	–	**0.12**	**0.12**	**0.12**	0.13	0.14	–	343	343	**276**	**276**	**276**
37	–	0.07	**0.06**	0.08	0.09	0.10	–	143	143	**114**	**114**	**114**
38	–	–	–	**0.08**	0.09	0.11	–	–	–	**128**	**128**	**128**
39	–	0.06	**0.05**	0.07	0.08	0.09	–	168	168	**120**	**120**	**120**
44	**0.42**	77.12	62.55	0.95	0.90	0.95	21475	1×10^5	91304	1549	**1422**	1449
45	23.10	–	–	**1.72**	1.83	1.95	2×10^6	–	–	3461	3402	**3287**
iii	**0.01**	2.39	2.30	0.18	0.20	0.08	**69**	22721	22721	746	835	356
vi	0.49	–	–	**0.16**	0.20	0.20	99594	–	–	**543**	677	630
vii	20.34	**0.38**	0.66	0.39	0.41	0.39	4×10^6	2232	3276	1027	1071	**981**

Table 1. Multidimensional knapsack problem experiments. Arabic and Roman numerals denote Beasley's and Petersen's instances. Instances solved within 2 seconds by all methods are omitted. The symbol – indicates that the 300-second timeout is reached.

threshold is $\tau = \frac{1}{n}\sum_{i=1}^{n} v_i$ (the average of the values). minItX is the number of steps prior to the threshold test. A 300-second timeout is set.

We used two models to encode the problem. $\boldsymbol{X}$ is a vector of Boolean variables, P is an integer variable with $\text{dom}(P) = [0, \sum_{i=1}^{n} v_i]$.

$$\text{Maximize } P \qquad \text{(Model 1)}$$
$$\text{MK}(\boldsymbol{X}, P, \boldsymbol{v}, \text{W}, \boldsymbol{c})$$

$$\text{Maximize } P \qquad \text{(Model 2)}$$
$$P = \langle \boldsymbol{v}, \boldsymbol{X} \rangle \,; \langle \boldsymbol{W_{j*}}, \boldsymbol{X} \rangle \leq c_j, \forall j$$

Model 1 contains a single MK constraint. Model 2 decomposes the MK constraint into m linear inequalities. Solvers usually enforce bound consistency on the constraints of Model 2. We also implemented Trick [21]'s propagator that enforces domain consistency. We compare ten approaches :

- LIN: Model 2 with bound consistency enforced;
- DC: Model 2 with domain consistency enforced;
- LR: Model 1, the standard Lagrangian relaxation proposed by Cambazard and Fages [6], with maxItX = 0, i.e., without local alterations;
- LR+0: Model 1, the Lagrangian relaxation with the local addition proposed by Berthiaume and Quimper [3], with minItX = 0, i.e., without additional steps before the threshold test;
- PROJ+1, PROJ+2, PROJ+3: Model 1, our contribution, with minItX $\in \{1, 2, 3\}$ with projections on both sides (the quadratic program);
- LR+1, LR+2, LR+3: Model 1, our contribution, with minItX $\in \{1, 2, 3\}$ without projections on both sides (no quadratic program).

Table 1 presents the times to solve Beasley's instances and Petersen's instances and the node count. The DC approach was discarded from the table as it reaches the timeout too often without finding any feasible solution. PROJ+$\{1, 2, 3\}$ were discarded because in average they were at least 1921% slower than LR+$\{1, 2, 3\}$. Instances solved in 2 seconds by all the remaining six approaches were omitted. Table 1 shows that our methods LR+$\{1, 2, 3\}$ are significantly faster than LIN, LR, and LR+0. There are 11 (10) instances solved

under one second by all of our methods while LIN (LR and LR+0) reached the timeout.

The variation between the columns LR+$\{1, 2, 3\}$ in Table 1 is at most 25 seconds (for instance 8); otherwise they are in the range of 1 second.

All methods LR+$\{1, 2, 3\}$ close instances with fewer nodes than all other methods except for instance (iii); in some cases *several* orders of magnitude. Among the methods LR+$\{1, 2, 3\}$, the number of nodes is generally in the same order of magnitude. We can see a small correlation with the number of nodes and the time to solve an instance; generally, the fastest method also develops fewer nodes. From the node count, we see that Lagrangian methods explore a completely different search tree than the LIN model.

We tested six different thresholds to study the impact of this parameter on the performance of the methods LR+$\{1, 2, 3\}$; $\tau \in \{10^{-2}, 10^{-1}, 10^{0}, 10^{1}, 10^{2}, 10^{3}\}$. We compared the average of the average time differences between Table 1 and the thresholds studied. On average, with the threshold chosen in Table 1, LR+1 is 29.31 seconds faster with a standard deviation of 84.12 seconds. LR+2 is 0.56 seconds faster with a standard deviation of 3.80 seconds. LR+3 is 0.22 seconds faster with a standard deviation of 0.99 seconds. LR+1 is much more sensitive to threshold variations than LR+$\{2, 3\}$. We experimentally see that when more additional steps are considered, the more robust the algorithm becomes.

The lines in Table 2 correspond to the nine problem classes from Chu and Beasley [7]. The entries are the number of times a given method finds the best objective value within the 300-second timeout. Except for instances with 5 resources and 100 items, and 10 resources and 100 items, all methods reach the timeout. Table 2 shows that for all instance classes (except the 5 resources and 250 items) LR+$\{1, 2, 3\}$ find the best objective value within the timeout.

4.2 Uncapacitated Facility Location Problem

In the uncapacitated facility location problem, there is a set of facility locations $\mathcal{L}$ and a set of cities $\mathcal{C}$. The goal is to choose which facility to build, and, among these, which to connect to every city to supply them while minimizing the transportation cost and building cost. Let $c_{i,j}$ be the transportation cost between the facility location $i \in \mathcal{L}$ and the city $j \in \mathcal{C}$, and let b_i be the building cost of

# Res.	# items	LIN	LR	LR+0	LR+1	LR+2	LR+3
5	100	0	26	25	**30**	**30**	**30**
5	250	0	**12**	4	6	9	7
5	500	1	11	3	**14**	11	8
10	100	0	5	3	16	17	**18**
10	250	1	6	5	12	**17**	**17**
10	500	3	6	3	**14**	12	12
30	100	1	5	7	19	**23**	19
30	250	4	10	8	**19**	17	18
30	500	4	10	9	**19**	17	17

Table 2. The number of times that a given method provides the best solution within a time out of five minutes for Chu and Beasley [7]'s instances.

the facility $i \in \mathcal{L}$. $x_{i,j}$ is a binary variable that encodes if the facility $i \in \mathcal{L}$ is connected to the city $j \in \mathcal{C}$, and y_i is a binary variable that encodes if a facility is built at the location $i \in \mathcal{L}$. The problem is formulated as

$$\begin{array}{ll} \min & \sum_{i\in\mathcal{L}} \sum_{j\in\mathcal{C}} c_{i,j} x_{i,j} + \sum_{i\in\mathcal{L}} b_i y_i \\ \text{s.t.} & \sum_{i\in\mathcal{L}} x_{i,j} \geq 1, \quad \forall j \in \mathcal{C}, \quad x_{i,j} \leq y_i, \quad \forall i \in \mathcal{L}, \forall j \in \mathcal{C}. \end{array}$$

We tested Kuehn and Hamburger [13] benchmark and observed that all Lagrangian based method achieve the same filtering, we do not present these results.

We generated 24 instances with 47 cities from France and 55 cities from the United Kingdom, from which we randomly selected $|\mathcal{L}| \in \{5, 8\}$ facility locations and let the remaining cities form the set $\mathcal{C}$. The cost $c_{i,j}$ is the distance (in km) as a crow flies between the facility location and the city. For France (UK), two instances are created, one where all building costs are set to 1500 (150) and one where they are all set to 3000 (500). We set the timeout to 600 seconds.

In Table 3, only the 18 instances where at least one method closed the instance within the ten-minute timeout are presented; the other 6 instances are presented in Table 4. The method LR+3 is the fastest method for six instances and LR+2 for five instances. When LR is faster, the number of nodes is the same for every method, see Table 3, which indicates that LR+$\{0, 1, 2, 3\}$ do the same filtering as LR. On two occasions LR+3 is the only method to close the instance.

For instances where all methods reached the timeout, we report in Table 4 the value of the best solution found. We see that on two instances, LR finds the best solution. The difference, when LR is better than our methods, is smaller than when our methods are better. All methods are equivalent for two instances.

5 Analysis

In both experiments, all LR+$\{1, 2, 3\}$ outperform the traditional Lagrangian method and also the other local alterations method. Compared with the tradi-

n	m	Instance Name	LR	LR+0	LR+1	LR+2	LR+3
5	42	france_0_1500	60.61	62.35	184.11	60.51	**24.37**
		france_1_1500	317.70	148.10	283.72	**65.54**	90.32
		france_2_1500	264.70	217.23	**52.80**	53.62	84.98
		france_0_3000	273.51	191.56	145.62	**96.88**	118.50
		france_2_3000	117.09	167.47	149.73	104.44	**66.25**
	50	UK_0_150	582.60	–	86.15	38.37	**35.15**
		UK_1_150	213.68	220.50	56.92	31.09	**22.52**
		UK_2_150	–	–	–	–	**386.12**
		UK_1_500	238.12	197.50	**53.82**	121.31	269.04
8	39	france_0_1500	137.57	110.62	175.30	**70.28**	112.50
		france_1_1500	**0.30**	0.33	0.76	1.19	1.63
		france_2_1500	177.97	208.40	194.94	**68.35**	69.18
		france_0_3000	–	–	–	**343.54**	422.24
	47	UK_0_150	–	–	–	–	**596.18**
		UK_1_150	**0.52**	0.58	1.32	2.08	2.83
		UK_2_150	**0.95**	1.01	1.60	2.21	2.81
		UK_0_500	**0.49**	0.53	1.12	1.71	2.29
		UK_2_500	**0.48**	0.54	1.28	2.03	2.78

n	m	Instance Name	LR	LR+0	LR+1	LR+2	LR+3
5	42	france_0_1500	33572	29453	73039	21478	***7762***
		france_1_1500	145286	64671	114199	***24134***	31442
		france_2_1500	119090	93385	**21810**	*18849*	24940
		france_0_3000	135362	88490	60524	***34874***	37838
		france_2_3000	56282	71914	61150	39446	***23643***
	50	UK_0_150	231106	–	28317	11999	***10412***
		UK_1_150	84805	78840	19126	9875	***7030***
		UK_2_150	–	–	–	–	***124924***
		UK_1_500	104836	77681	***18819***	36184	78545
8	39	france_0_1500	44354	34193	48717	***18052***	27830
		france_1_1500	**322**	*322*	*322*	*322*	*322*
		france_2_1500	54000	58877	52922	**17590**	*17185*
		france_0_3000	–	–	–	***90339***	107511
	47	UK_0_150	–	–	–	–	***118982***
		UK_1_150	***386***	*386*	*386*	*386*	*386*
		UK_2_150	***344***	*344*	*344*	*344*	*344*
		UK_0_500	***342***	*342*	*342*	*342*	*342*
		UK_2_500	***386***	*386*	*386*	*386*	*386*

Table 3. Time (in secondes) (left) and number of nodes visited (right) to solves instances of the UFLP for France and UK with a timeout of 600 seconds.

n	m	Instance	LR	LR+0	LR+1	LR+2	LR+3
5	42	france_1_3000	**23551**	23612	23632	23612	23612
	50	UK_0_500	6501	6546	6365	**6183**	6726
		UK_2_500	8015	8015	8018	8022	**7319**
8	39	france_1_3000	**22993**	**22993**	**22993**	**22993**	**22993**
		france_2_3000	**23804**	**23804**	**23804**	**23804**	**23804**
	47	UK_1_500	**5804**	5837	5954	5893	5971

Table 4. Cost of the solution for instances that reached the timeout of 600 seconds.

tional Lagrangian method, this performance difference indicates that some filtering zones were missed while developing the sequence of Lagrange multipliers. However, the local alteration method LR+0 performs similarly to the traditional Lagrangian method. Hence, not all local alterations method aid for the overall performance. Even adding one step before the threshold test significantly helps the time performance; more specifically with the multidimensional knapsack problem. However, adding two or three steps makes the algorithm more robust to threshold variations, as we hypothesized at the end of Section 3.2. With the uncapacitated facility location problem, the speed up happened after adding two and three steps. The reason for these differences might be the entries of the weight matrix. In the multidimensional knapsack problem, $w_{ji} \in \mathbb{N}$ with no specific structure versus the uncapacitated facility location problem, where $w_{ji} \in \{\pm 1, 0\}$ and with a specific structure. The position and shape of the filtering and threshold zones vary drastically. Still, the performance differences of methods LR+$\{1, 2, 3\}$ with the local alterations method LR+0 are mainly a consequences of Theorems 1 and 2. We tested all the instances, and every underlying linear program optimal solutions had some non-integer variables. This indicates that $\boldsymbol{\lambda}^\star$ was always at the intersection of decision lines. Due to Theorem 2, we know that we must move away from $\boldsymbol{\lambda}^\star$ to get a relevant threshold test.

We observed a significant decrease in the number of nodes explored in both problems. The method LR has no guarantee to encounter all the filtering zones encountered by methods LR+$\{1, 2, 3\}$. The method LR+0 additional explorations depend completely on the threshold test, which has been shown to be a poor test in general. Because the multidimensional knapsack problem and the uncapacitated facility location problem aver very different problems, we conjecture that LR and LR+$\{1, 2, 3\}$ behave similarly on other problems.

6 Conclusion

We formalized some important observations in cost-based Lagrangian filtering theory. We designed a cost-based filtering algorithm based on a Lagrangian relaxation with enhanced filtering, which comes from the careful addition of a local alterations phase, for the MK constraint. Our methods have proven to be faster, and found better solutions within a timeout than the state-of-the-art filtering algorithms on two well-known problems.

References

1. Bajgiran, O.S., Cire, A.A., Rousseau, L.M.: A first look at picking dual variables for maximizing reduced cost fixing. In: Integration of AI and OR Techniques in Constraint Programming (2017)
2. Benchimol, P., Hoeve, W.J.v., Régin, J.C., Rousseau, L.M., Rueher, M.: Improved filtering for weighted circuit constraints. Constraints (2012)
3. Berthiaume, F., Quimper, C.G.: Local alterations of the lagrange multipliers for enhancing the filtering of the atmostnvalue constraint. In: International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research (2024)
4. Bessa, S., Dabert, D., Bourgeat, M., Rousseau, L.M., Cappart, Q.: Learning valid dual bounds in constraint programming: Boosted lagrangian decomposition with self-supervised learning. In: Proceedings of the 39th AAAI Conference on Artificial Intelligence and 37th Conference on Innovative Applications of Artificial Intelligence and 15th Symposium on Educational Advances in Artificial Intelligence (2025)
5. Boudreault, R., Quimper, C.G.: Improved cp-based lagrangian relaxation approach with an application to the tsp. In: Proceedings of the 30th International Joint Conference on Artificial Intelligence (2021)
6. Cambazard, H., Fages, J.G.: New filtering for atmostnvalue and its weighted variant: A lagrangian approach. Constraints (2015)
7. Chu, P.C., Beasley, J.E.: A genetic algorithm for the multidimensional knapsack problem. Journal of Heuristics (1998)
8. Fahle, T., Sellmann, M.: Cost based filtering for the constrained knapsack problem. Annals of Operations Research (2002)
9. Focacci, F., Lodi, A., Milano, M.: Cost-based domain filtering. In: Principles and Practice of Constraint Programming (1999)
10. Held, M., Karp, R.M.: The traveling-salesman problem and minimum spanning trees. Operations Research (1970)
11. Isoart, N., Régin, J.C.: Adaptive cp-based lagrangian relaxation for tsp solving. In: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (2020)
12. Katriel, I., Sellmann, M., Upfal, E., Van Hentenryck, P.: Propagating knapsack constraints in sublinear time. In: Proceedings of the 22nd National Conference on Artificial Intelligence (2007)
13. Kuehn, A.A., Hamburger, M.J.: A heuristic program for locating warehouses. Management Science (1963)
14. Lu, Y., Vasko, F.J.: A comprehensive empirical demonstration of the impact of choice constraints on solving generalizations of the 0–1 knapsack problem using the integer programming option of cplex®. Engineering Optimization (2020)
15. Petersen, C.C.: Computational experience with variants of the balas algorithm applied to the selection of r&d projects. Management Science (1967)
16. Peterson, A.: ojalgo: Open-source java tools for linear and mathematical programming. http://ojalgo.org (2003)
17. Polyak, B.T.: Minimization of unsmooth functionals. USSR Computational Mathematics and Mathematical Physics (1969)
18. Prud'homme, C., Fages, J.G.: Choco-solver: A java library for constraint programming. Journal of Open Source Software (2022)

19. Sellmann, M.: Theoretical foundations of cp-based lagrangian relaxation. In: Principles and Practice of Constraint Programming (2004)
20. Sellmann, M., Fahle, T.: Constraint programming based lagrangian relaxation for the automatic recording problem. Annals of Operations Research (2003)
21. Trick, M.A.: A dynamic programming approach for consistency and propagation for knapsack constraints. Annals of Operations Research (2003)